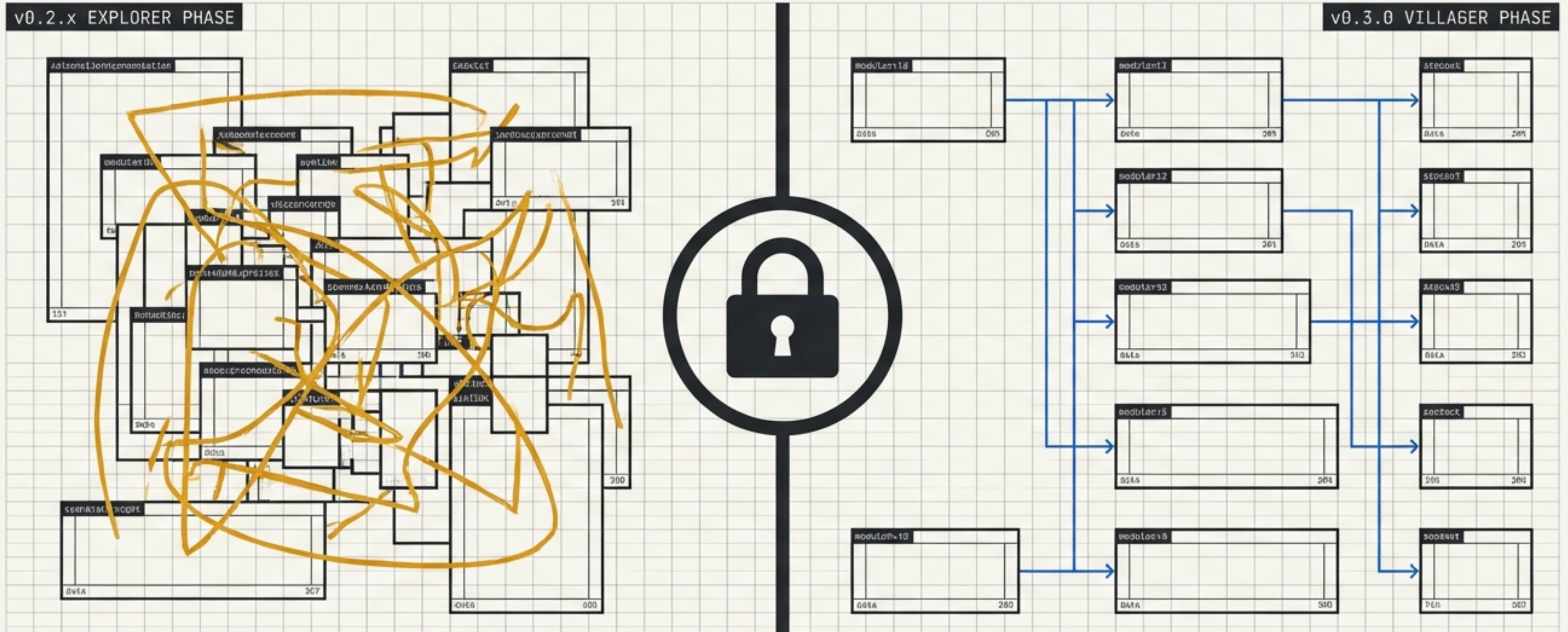
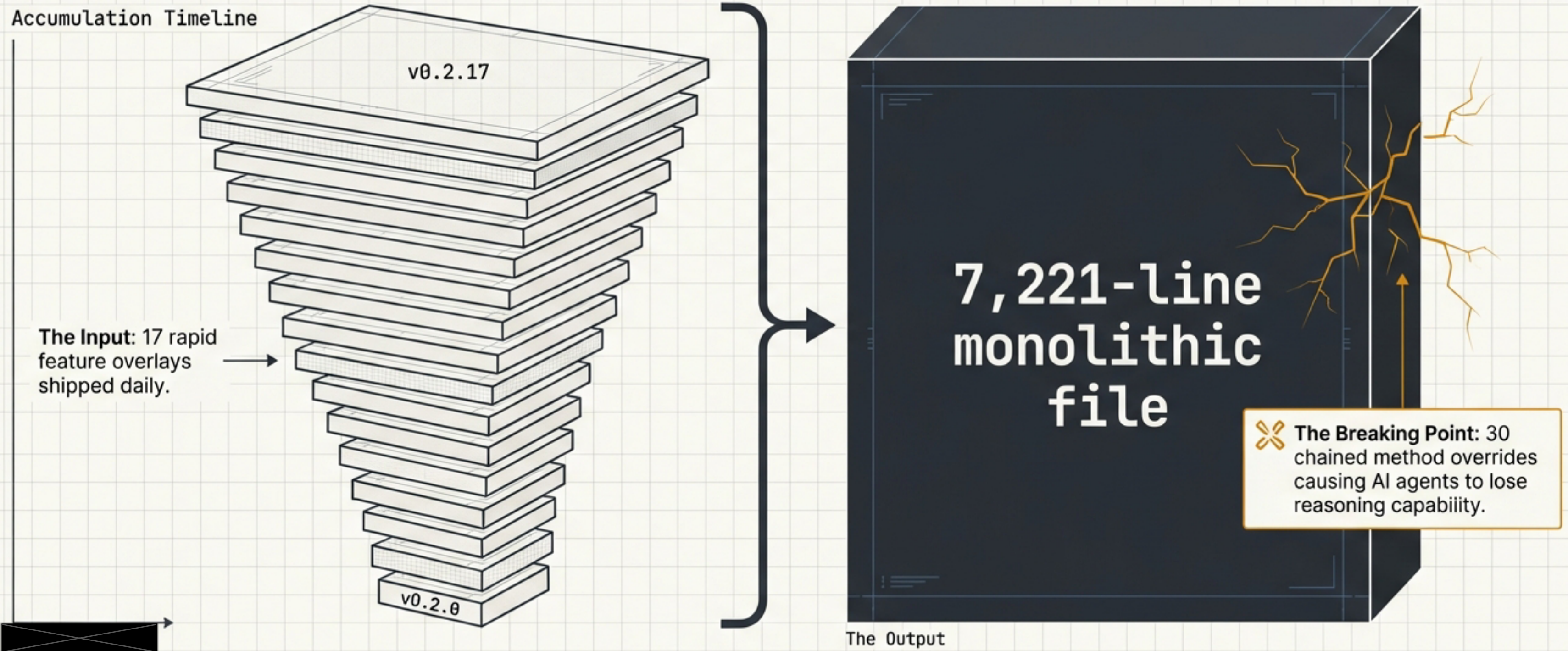


The Villager Phase

Taking strict control of the code your AI agents wrote.



Rapid AI iteration creates technical debt that eventually chokes both humans and LLMs.

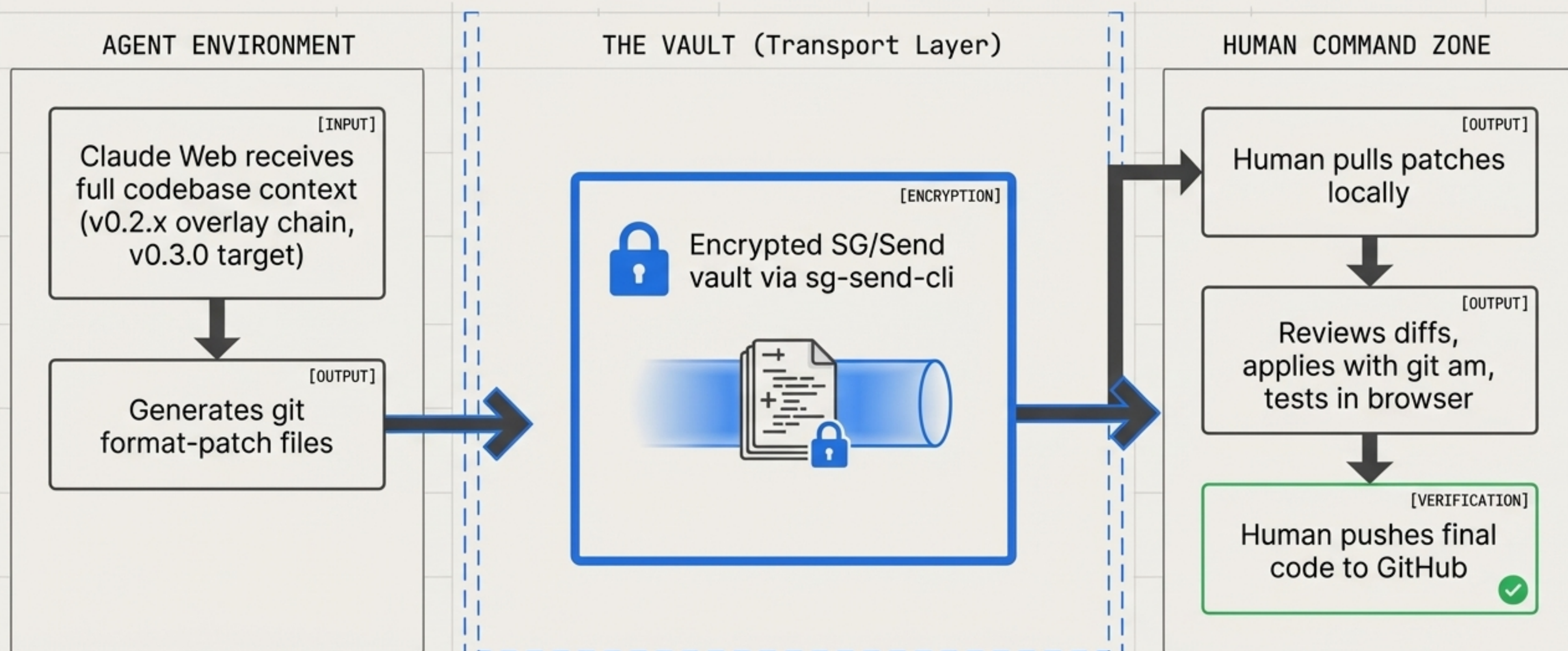


Refactoring a 7,000-line monolith breaks standard AI tooling.

Diagnostic Comparison Matrix

Agent / Tool	Execution Result	Architectural Verdict
OpenAI Codex	[FAILED]	Not ready for scale or whole-codebase architectural reasoning.
Claude Code	[TIMED OUT]	Excellent for focused execution (one change at a time). Failed after 232 tool calls; unable to hold entire codebase context across dozens of files.
Claude Web	[BREAKTHROUGH]	No direct Git access, but possesses massive context windows for architectural conversation and produces highly accurate patch files.

The Vault Workflow physically isolates the AI agent from the repository



Patch-based transport is the definitive blueprint for secure AI integration.



Provenance

Every change is traced, signed, and versioned.



Blast Radius

Bounded diffs prevent rogue modifications and accidental commits.



Multi-agent Review

QA and AppSec agents review patches before they touch the actual codebase.



Identity Separation

Agents operate without your Git credentials, SSH keys, or repo write access.



Offline Capability

Uncoupled from specific CI/CD pipelines; vaults can be transferred via any channel.

Effective agentic workflows require operating at two distinct altitudes simultaneously.

[ALTITUDE 1: THE HUMAN / VILLAGER]



Architectural Strategy

- Reading every line of the v0.2.x overlay chain
- Verifying method signatures
- Enforcing component boundaries
- Controlling execution order

[ALTITUDE 2: THE AGENT]



The Heavy Lifting

- Producing 16 complex patches
- Managing boilerplate
- Generating diffs

Automated tests will validate AI hallucinations. Reading the code is non-negotiable.

```
1 import crypto
2 import logging
3
4 class FriendlyCrypto:
5     def __init__(self):
6         self.logger = logging.getLogger('friendly_crypto')
7
8     def resolve(self, token):
9         # AI hallucinated this method
10        # Should not exist
11        pass
12
13 def process_token(token):
14     fc = FriendlyCrypto()
15     fc.resolve(token) # <-- Hallucinated Call
16
17 if __name__ == '__main__':
18     process_token('abc123xyz')
```

FriendlyCrypto.resolve()

[THREAT: P0 BUG]

The Threat

A P0 bug where an AI hallucinated an API call, breaking all friendly token URLs.

[DEFENCE: AGENT]

The Failure Point

Tests alone would miss this because the AI would write the tests against the same hallucinated API.

[DEFENCE: P0 BUG]

A biles reading the raw code recognises that FriendlyCrypto.resolve() is not.

The Defence

A human reading the raw code recognises that FriendlyCrypto.resolve() is not a real method.

[DEFENCE: HUMAN REVIEW]

The Villager phase fundamentally resets the architectural baseline.

ARCHITECTURE

1 monolithic file → **25 modular files**

7,221 lines → 6,058 lines

COMPONENTS

Tangled overrides → **7 distinct modules**

Plus 6 Shadow DOM components

TECHNICAL DEBT

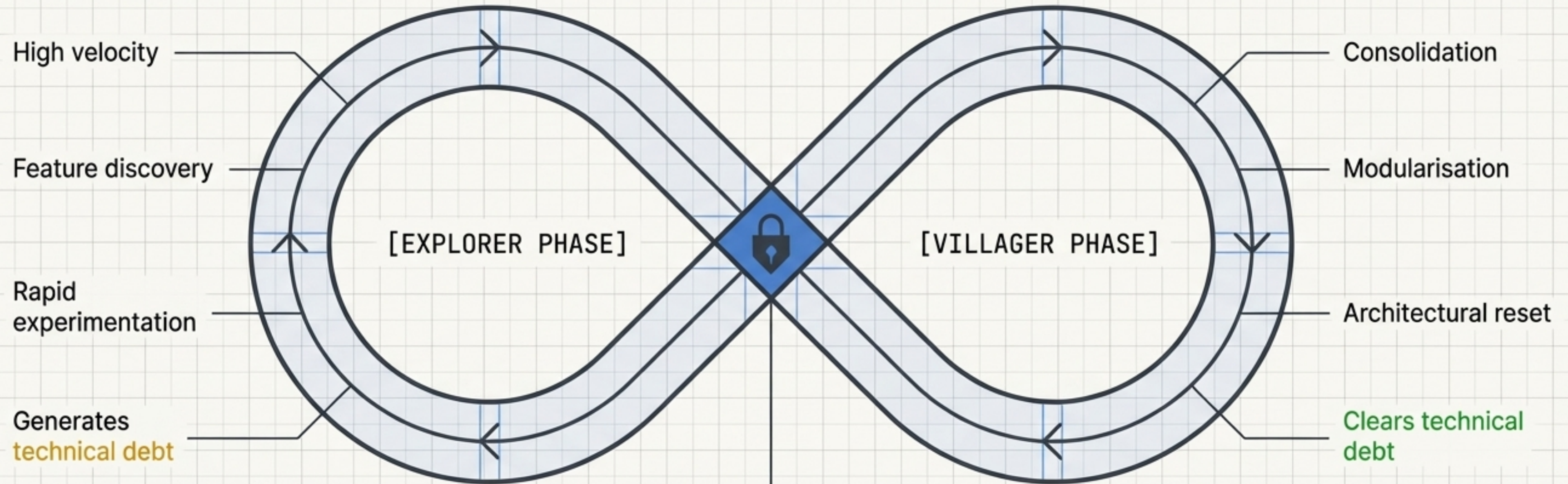
27 **external symlinks** → **0 external symlinks**

PROCESS EFFICIENCY



CI pipeline size reduced by 50%

Taking time to consolidate accelerates the next phase of innovation.

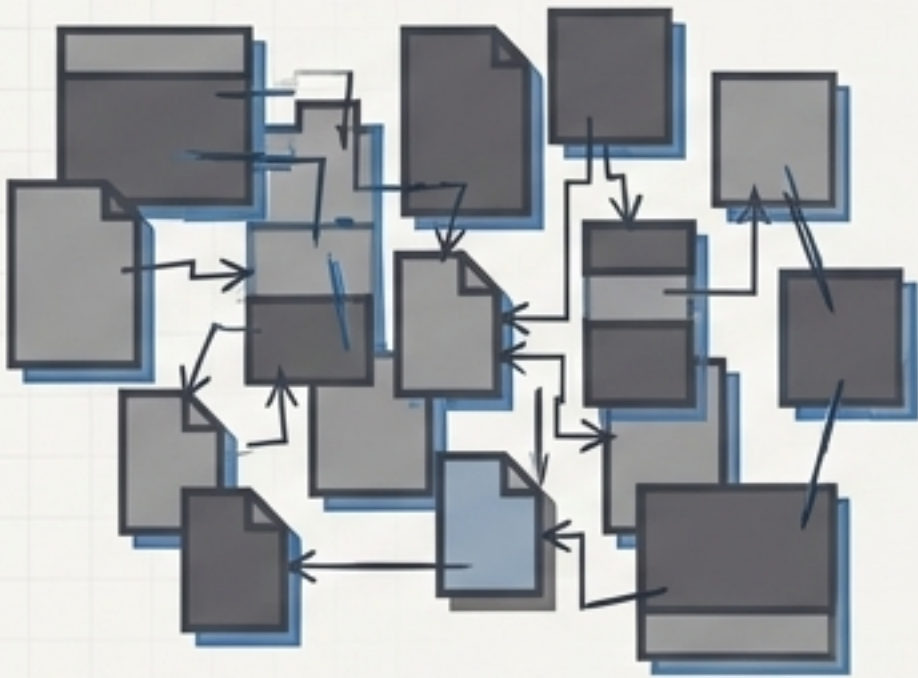


[CORE INSIGHT]

Investing in the Villager phase raises the structural floor, allowing the subsequent Explorer phase to move significantly faster on a clean foundation.

Iterative Flow Development must evolve to support multi-agent scaling.

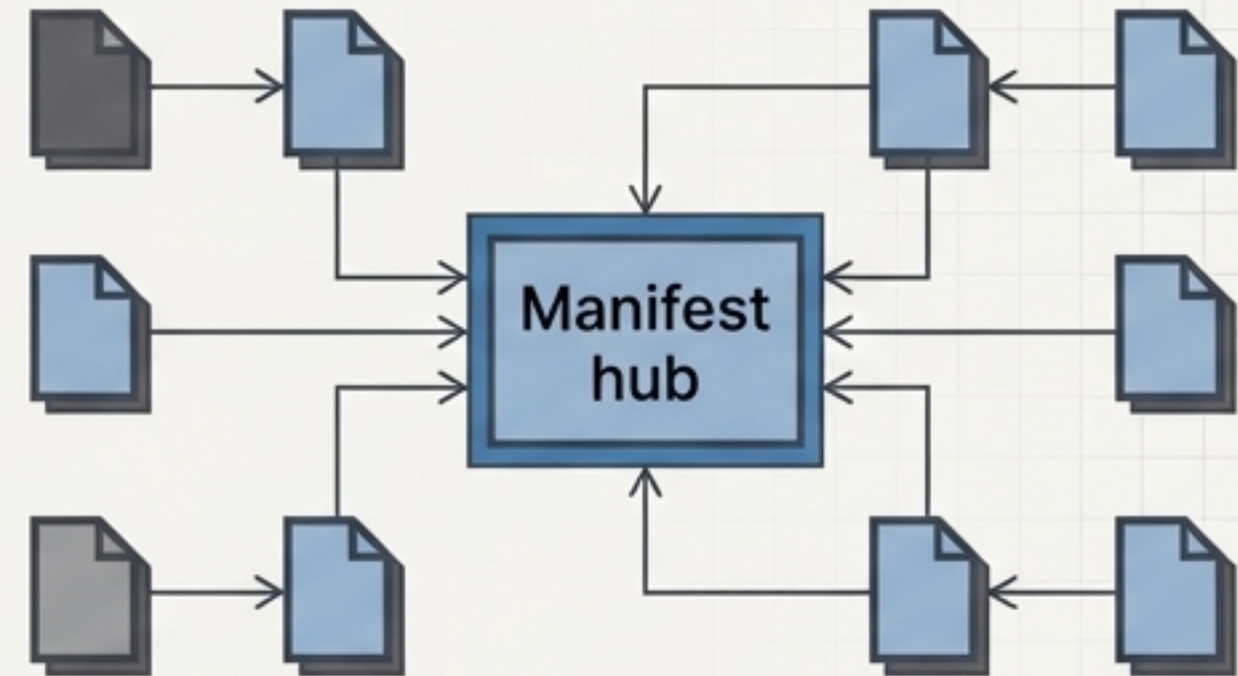
[v1: THE OLD PATTERN]



- Method-level patching.
- Hardcoded script tags.

⚠ Result: Brittle, monolith-prone.

[v2: THE NEW STANDARD]



- File-level replacement.
- Explicit dependency manifests.
- Dynamic loaders reading from manifests.

✓ Result: Scalable, agent-friendly architecture.

The secure blueprint for multi-agent software development.



Isolate Access

Never give AI agents direct write access to the production repository.

Deploy Vaults

Adopt the encrypted Vault & Patch transport pattern for all AI-generated code.

Embrace Cycles

Systematise the Explorer (velocity) and Villager (consolidation) phases.

Read the Code

Mandate human oversight at the hands-on architectural altitude to catch systemic hallucinations.